

Advanced SQL Interview Guide

Comprehensive Solutions & Detailed Interview Frameworks for Technical Professionals

Part A: Descriptive Questions & Interview Explanations

1. What is the difference between a Primary Key, Unique Key, and Foreign Key? Explain with suitable examples.

Core Answer:

- **Primary Key:** Uniquely identifies each record in a table. It cannot contain **NULL** values, and a table can have only one primary key.
- **Unique Key:** Ensures all values in a column are unique. Unlike a primary key, it allows a single **NULL** value (depending on the RDBMS), and a table can have multiple unique keys.
- **Foreign Key:** Establishes a link between two tables by referencing the primary key of another table, ensuring referential integrity.

 **Interview Tip:** "When explaining this in an interview, emphasize the real-world constraint rule: *Primary Key = Identity + Integrity*, while *Foreign Key = Relationship bridge*."

2. What are SQL Constraints? Explain NOT NULL, CHECK, DEFAULT, UNIQUE, and PRIMARY KEY with real-world use cases.

Core Answer:

SQL constraints are rules enforced on data columns in a table. They ensure data accuracy and reliability.

- **NOT NULL:** Prevents null entries (e.g., employee name cannot be blank).
- **CHECK:** Ensures values meet a specific condition (e.g., *age >= 18*).
- **DEFAULT:** Assigns a default value if none is provided (e.g., status defaults to 'Active').
- **UNIQUE:** Guarantees all values in a column are distinct (e.g., email address).
- **PRIMARY KEY:** Combines **NOT NULL** and **UNIQUE** uniqueness for row identification.

 **Interview Tip:** Mention that applying constraints at the database level is a best practice because it stops corrupted data before it touches application logic.

3. What is the difference between DELETE, TRUNCATE, and DROP commands? When should each be used?

Core Answer:

- **DELETE** is a DML command used to remove specific rows based on a **WHERE** clause. It logs individual row deletions, fires triggers, and can be rolled back.
- **TRUNCATE** is a DDL command that removes all rows from a table quickly by deallocating pages. It cannot use a **WHERE** clause, does not fire triggers, and resets identity seeds.
- **DROP** is a DDL command that completely removes the table structure and its data from the database.

 **Interview Tip:** Interviewers love asking this to test safety awareness. Always state: "Use **DELETE** when conditional removal is needed with rollback capability; use **TRUNCATE** for fast table resets; use **DROP** only when decommissioning a table structure entirely."

4. What are Aggregate Functions in SQL? Explain COUNT(), SUM(), AVG(), MIN(), and MAX() with examples.

Core Answer:

Aggregate functions perform a calculation across multiple rows of a single column and return a single scalar value. They are almost always used with the **GROUP BY** clause.

- **COUNT(*)**: Counts total rows.
- **SUM(col)**: Calculates total sum of numeric column.
- **AVG(col)**: Computes mathematical average.
- **MIN() / MAX()**: Finds lowest and highest values.

 **Interview Tip:** Highlight that aggregate functions automatically ignore **NULL** values, except for **COUNT(*)** which counts all rows including nulls.

5. What is the difference between WHERE and HAVING clauses? Explain with examples.

Core Answer:

- **WHERE** filters individual rows *before* any grouping or aggregation takes place. It cannot contain aggregate functions.
- **HAVING** filters groups *after* the aggregation has been computed using **GROUP BY**. It can contain aggregate functions.

 **Interview Tip:** Explain the execution sequence: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. This clearly answers *why* WHERE cannot evaluate aggregate functions.

6. What are Joins in SQL? Explain INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN with practical scenarios.

Core Answer:

Joins combine rows from two or more tables based on a related column between them.

- **INNER JOIN:** Returns records that have matching values in both tables.
- **LEFT JOIN:** Returns all records from the left table, and matched records from the right table (unmatched right side evaluates to **NULL**).
- **RIGHT JOIN:** Returns all records from the right table, and matched records from the left table.
- **FULL OUTER JOIN:** Returns all records when there is a match in either left or right table.

 **Interview Tip:** Use a real-world context like Customers and Orders to explain left/inner joins clearly.

7. What is a Subquery? Explain the advantages and disadvantages of using subqueries.

Core Answer:

A subquery is a query nested inside another SQL query (such as inside a **SELECT**, **FROM**, or **WHERE** clause).

- **Advantages:** Improves readability, breaks down complex logic, and allows structured step-by-step filtering.
- **Disadvantages:** Can sometimes cause performance overhead on large datasets compared to JOIN equivalents if not properly indexed.

 **Interview Tip:** Mention correlated subqueries vs non-correlated subqueries, noting that correlated subqueries execute once for every row processed by the outer query.

8. What is a User Defined Function (UDF)? Explain the types of UDFs and their practical applications.

Core Answer:

A User-Defined Function is a routine accepted by the database engine that accepts parameters, performs calculations or business logic, and returns a result. Types include scalar functions (returning single values) and table-valued functions (returning result sets).

 **Interview Tip:** Keep it concise: UDFs encapsulate reusable modular business logic directly inside database queries.

9. What is the difference between a Stored Procedure and a Function in SQL?

Core Answer:

- **Function:** Must return a value, can be used inside SQL statements (**SELECT**, **WHERE**), and cannot perform DML modifications (insert/update/delete state changes in tables).
- **Stored Procedure:** Can return multiple output parameters or no value, cannot be embedded directly inside a **SELECT** statement, and can handle complex transactions and DML operations.

 **Interview Tip:** "Use a function when you need calculations to plug into queries; use a stored procedure when executing workflows, transactions, or batch DML."

10. What is Normalization? Explain 1NF, 2NF, and 3NF with examples.

Core Answer:

Normalization is the process of organizing database tables to minimize redundancy and dependency.

- **1NF:** Ensures atomic values (no repeating groups or comma-separated lists in a column).
- **2NF:** Must be in 1NF and eliminate partial functional dependencies (all non-key columns must depend entirely on the primary key).
- **3NF:** Must be in 2NF and eliminate transitive dependencies (non-key columns must depend only on the primary key, not on other non-key columns).

 **Interview Tip:** Mention that normalization reduces data anomalies during insert, update, and delete operations.

Part B: Practical Hard-Level Questions & Solutions

Question 1: Find Employee(s) Having the Second Highest Salary

To solve this robustly across different SQL engines without hardcoding row positions, use subqueries or window functions:

```
SELECT emp_id, emp_name, department, salary, joining_date
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
    WHERE salary < (SELECT MAX(salary) FROM employees)
);
```

 **Interview Tip:** Alternatively, mention the DENSE_RANK() window function approach: *SELECT * FROM (SELECT *, DENSE_RANK() OVER(ORDER BY salary DESC) as rnk FROM employees) t WHERE rnk = 2;.*

Question 2: Create a Function `bonus\_amount()` Based on Salary Thresholds

Implementation using MySQL syntax with conditional logic:

```
DELIMITER //
CREATE FUNCTION bonus_amount(emp_salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE bonus DECIMAL(10,2);
    IF emp_salary >= 100000 THEN
        SET bonus = emp_salary * 0.20;
    ELSEIF emp_salary >= 70000 THEN
        SET bonus = emp_salary * 0.15;
    ELSEIF emp_salary >= 50000 THEN
        SET bonus = emp_salary * 0.10;
    ELSE
        SET bonus = emp_salary * 0.05;
    END IF;
    RETURN bonus;
END //
DELIMITER ;

-- Query to display required details:
```

```
SELECT emp_id, emp_name, salary, bonus_amount(salary) AS bonus_amount
FROM employees;
```

 **Interview Tip:** Explain the *DETERMINISTIC* keyword, which tells the optimizer whether the function always returns the same output for identical inputs.

Question 3: Students Enrolled in the Course Having the Highest Fees

Using subqueries to find the maximum fee course and fetching enrolled students:

```
SELECT s.roll_no, s.student_name, c.course_name, c.fees
FROM students s
JOIN courses c ON s.course_id = c.course_id
WHERE c.fees = (SELECT MAX(fees) FROM courses);
```

 **Interview Tip:** Note how joining the tables first allows clean filtering using a subquery on the aggregate maximum fee.

Question 4: Categories with Total Sales Greater Than Overall Average Category Sales

Using a Common Table Expression (CTE) to aggregate category sales and compare against the average of category sums:

```
WITH CategorySales AS (
    SELECT category, SUM(quantity * price) AS total_sales
    FROM sales
    GROUP BY category
)
SELECT category, total_sales
FROM CategorySales
WHERE total_sales > (SELECT AVG(total_sales) FROM CategorySales);
```

 **Interview Tip:** Using CTEs makes complex multi-step analytical queries readable and maintainable compared to nested subqueries.

Question 5: Performance Grade Function and Analytics

Creating the performance grade function and executing the analytical queries:

```
DELIMITER //
```

```
CREATE FUNCTION performance_grade(marks INT)
RETURNS VARCHAR(30)
DETERMINISTIC
BEGIN
    DECLARE grade VARCHAR(30);
    IF marks >= 90 THEN
        SET grade = 'Excellent';
    ELSEIF marks >= 75 THEN
        SET grade = 'Good';
    ELSEIF marks >= 60 THEN
        SET grade = 'Average';
    ELSE
        SET grade = 'Needs Improvement';
    END IF;
    RETURN grade;
END //
```

```
DELIMITER ;
```

```
-- 1. Display all student details with grade:
SELECT student_id, student_name, marks, performance_grade(marks) AS grade FROM students;
```

```
-- 2. Count how many students belong to each grade category:
SELECT performance_grade(marks) AS grade, COUNT(*) AS student_count
FROM students
GROUP BY grade;
```

```
-- 3. Display the grade category having the maximum number of students:
SELECT performance_grade(marks) AS grade, COUNT(*) AS student_count
FROM students
GROUP BY grade
ORDER BY student_count DESC
LIMIT 1;
```



Interview Tip: Explain how **GROUP BY** combined with sorting and **LIMIT 1** efficiently answers ranking or maximum threshold questions.